

Sysmex CS-2000i: руководство по сервису

Архитектура, последовательность вызовов, хранение сырых результатов и эксплуатация. Состояние реализации: 27 сентября 2026 г.; .NET 9.0.

1. Назначение и состав решения

Windows Service принимает TCP-подключение анализатора Sysmex CS-2000i. В решение входят два отдельных протокольных драйвера: ASTM E1381/E1394 и Sysmex Host Online. Подключаемые DLL реализуют общий контракт IAnalyzerDriver; параметры берутся только из JSON.

AnalyzerService.Host → Windows Service, конфигурация, загрузка DLL, жизненный цикл
 AnalyzerService.Contracts → IAnalyzerDriver, AnalyzerSettings, IAnalyzerLogger
 AnalyzerService.Transport → общий TCP-host, состояние listener и соединений
 AnalyzerService.LisDatabase → LisDBProvider, заказ и сопоставление кодов
 AnalyzerService.ResultFiles → RawResultQueue, файлы и выделенный поток
 SysmexCS2000.Driver → ASTM-кадры, сессия, разбор и формат ЛИС
 SysmexCS2000.HostOnline.Driver → R/S/D-тексты, сборка блоков, формат ЛИС

Новая очередь файлов намеренно отделена от TCP-host: транспорт знает только сокеты, а очередь знает только каталог, байты и функцию преобразования. Такая граница подходит и для будущих приборов с другим прикладным протоколом.

2. Запуск службы: точный порядок

1. Program создаёт Generic Host и регистрирует Worker, поставщик JSON, проверку настроек, AnalyzerManager и фабрику логгеров.
2. Worker.ExecuteAsync читает все JSON из каталога configs рядом с исполняемым файлом, проверяет каждую конфигурацию и добавляет активные приборы в AnalyzerManager.
3. AnalyzerManager запускает разрешённые приборы. AnalyzerRuntime.Load загружает указанную DIPath сборку через DriverLoadContext, создаёт IAnalyzerDriver и вызывает Initialize.
4. RunAsync драйвера создаёт RawResultQueue, при ResultHandlerStatus=true запускает его выделенный поток, затем вызывает TcpHost.Start(IPaddress, Port). Поток очереди сразу видит .raw от прошлых запусков.
5. TcpHost.AcceptAsync ожидает клиента без блокирования потока службы. После подключения драйвер читает сообщения прибора в цикле до отключения или остановки.

Если параметр WorkStatus или ActiveStatus отключает прибор, его запуск определяется существующей логикой менеджера. Новых ключей конфигурации для очереди не введено.

3. Приём и маршрутизация сообщений

Sysmex Host Online

1. ReadTextAsync принимает кадр между STX и ETX; AddBlock собирает все блоки одного сообщения и сохраняет исходные кадры без нормализации в Raw.
2. R221: ParseInquiry → LisDBProvider.GetOrder → BuildOrder → отправка S221 по TCP. Этот путь остаётся синхронным по отношению к запросу задания.
3. D121/D221: по Sample Distinction Code в позиции 8 определяется контроль C. Пациентский результат передаётся SaveResult, контроль - SaveQualityControl. Приёмник не вызывает TranslateResultCode и не создаёт файлы ЛИС.

4. DS21 представляет информацию о пробе и не создаёт файла результата.

ASTM

1. AstmSession.ReceiveMessageAsync принимает ENQ, проверяет кадры и checksum, отправляет предусмотренные ASTM ACK/NAK и завершает транзакцию на EOT. Возвращает текст принятых записей и исходные байты транзакции.

2. Q-запись: GetQuerySampleId → LisDBProvider.GetOrder → BuildOrder/BuildEmpty → AstmSession.SendMessageAsync.

3. O/R-записи: по Sample ID QC... или полю Action Code = Q определяется контроль. Далее исходные байты идут в SaveQualityControl либо SaveResult.

У Host Online в текущем режиме дополнительный прикладной ACK на D/S не вводился: решение о нём отложено до проверки с реальным анализатором. ASTM ACK/NAK относится к уже существующему канальному обмену ASTM и не менялся в этой доработке.

4. Файловая очередь результатов

Путь вычисляется из существующего ResultsFolder: абсолютный путь используется как есть; относительный разрешается относительно <каталог службы>/<AnalyzerName>. Каждая законченная передача результата одного образца создаёт отдельный файл. При повторном измерении идентификатор образца совпадает, но timestamp и GUID сохраняют оба измерения.

При ResultHandlerStatus=false сырые сообщения продолжают сохраняться, но рабочий поток не запускается. Они обрабатываются при следующем запуске с включённым обработчиком.

```
ResultsFolder/
SYS2000_<SampleId>_<UTC>_<GUID>.raw ← готовые пациентские сообщения
archive/ ← успешно преобразованные исходные .raw
errors/ ← ошибка разбора, нет результатов или кодов ЛИС
QualityControl/ ← неизменённые QC .raw; будущий обработчик
OutputFolder/ ← выходные .res и .ok для ЛИС
```

1. SaveResult синхронно пишет байты в .raw.tmp и переименовывает в .raw. Поток обработки не видит частично записанный файл.

2. Выделенный Thread запускает Run; он обходит *.raw верхнего уровня при старте и далее после сигнала записи или каждые две секунды. Папки archive/errors/QC не сканируются.

3. ProcessFile читает байты; драйвер восстанавливает тело протокола и вызывает свой ResultHandler. Тот извлекает тесты, сопоставляет их через LisDBProvider и создаёт .res, а затем маркер .ok в OutputFolder.

4. После успеха исходный .raw переносится в archive. Любое исключение обработки логируется с путём файла и ведёт к переносу в errors. Это касается пустого результата и случая, когда ни один тест не настроен для ЛИС.

5. Если сам перенос не удался, исходник остаётся на месте и ошибка логируется. Имя выходных .res/.ok основано на имени .raw; наличие .ok предотвращает повторный вывод того же результата при следующем проходе.

Важно: перемещение в errors применяется также к временным ошибкам БД ЛИС. Такие файлы можно проанализировать и после устранения причины вернуть вручную в корень ResultsFolder для повторной обработки. Перемещение выполнять только после проверки выходных .ok и журналов.

5. Контроль качества

Контроль сохраняется функцией SaveQualityControl непосредственно при приёме. В QualityControl лежат точные входящие байты: для Host Online - один или несколько кадров STX...ETX, для ASTM - ENQ, все входящие кадры и EOT. Поля не переводятся в формат ЛИС, коды тестов не сопоставляются, .res/.ok не создаются. Будущий QC-обработчик может читать эти .raw независимо от текущей очереди пациентов.

6. Остановка и восстановление

1. Service Control Manager вызывает Worker.StopAsync, затем AnalyzerManager.StopAllAsync.
2. AnalyzerRuntime отменяет token и вызывает IAnalyzerDriver.StopAsync; драйвер закрывает активный TcpClient и TcpHost, прерывая AcceptAsync/ReadAsync.
3. RunAsync выходит из сетевого цикла и в finally вызывает RawResultQueue.Stop. Выделенный поток завершает текущий файл и прекращает обход каталога.
4. После завершения задачи драйвера AnalyzerRuntime освобождает ресурсы и контекст DLL. Непереработанные .raw остаются на диске и подбираются при следующем запуске.

Асинхронность используется в ожидании сети и завершения рабочих задач. Краткие файловые операции и обращение к БД внутри выделенного обработчика выполняются синхронно; TCP-чтение не ждёт преобразования результата в ЛИС.

7. Настройка, сборка и проверка

Используется имеющийся JSON из configs. Для этой доработки ни один ключ не добавлен. Проверьте AnalyzerName, ActiveStatus, WorkStatus, ResultHandlerStatus, Protocol, DllPath, IPAddress, Port, ResultsFolder, OutputFolder и ConnectionString. Два драйвера не могут слушать одну пару IP-адрес/порт одновременно.

```
dotnet build .\AnalyzerService.sln --maxcpucount:1
dotnet run --project .\tests\SysmexCS2000.Driver.Tests
dotnet run --project .\tests\SysmexCS2000.HostOnline.Driver.Tests
.\package.ps1 # Windows: публикует host и DLL с зависимостями
```

После публикации проверьте наличие AnalyzerService.ResultFiles.dll в каталоге каждого драйвера. Для диагностики смотрите Service/Transport/Protocol/Result/Error логи: создание .raw, перенос в archive/errors и точное исключение каждого сбоя. Общий TcpHost периодически пишет состояние listener, endpoint, клиента, RX/TX и последнюю ошибку. Отсутствие RX при работающем listener не означает блокировку сокета.

8. Границы и источники

Тексты и каналные правила протоколов берутся из проектных документов 2_5197664667366890337.pdf (Host Online) и 2_5197664667366890338.pdf (ASTM). Это руководство описывает фактическую реализацию кода; оно не заменяет проверку обмена и ACK с реальным CS-2000i перед промышленным вводом.

Проверены сборка .NET 9 и консольные тесты обоих драйверов. Интеграционные испытания с реальным прибором и БД ЛИС требуют соответствующего стенда.